

LORENZ SYSTEM

The Lorenz System was developed by Edward Lorenz in 1963 as a simplified mathematical model for atmospheric convection. It exhibits chaotic behaviour, meaning it has a sensitive dependence on the initial conditions of the system. Even a small variation in the initial conditions of two separate systems leads to extreme variations with time.

With this mini project, I plan to enhance my understanding of this concept by simulating this system using c++ and using python for plotting the graphs.

The objective of this project is :

- Simulate the Lorenz attractor
- Compare two lorenz systems with different starting states and observe their divergence with time
- Compute the largest Lyapunov exponent using an algorithm presented by Benettin et al 1980.

$\lambda_1 \approx 0.9056$ (largest Lyapunov exponent),

$\lambda_2 \approx 0$,

$\lambda_3 \approx -14.5723$

RK4 Analysis

As errors compound quickly in a chaotic system, we need something that minimizes the error. Since the Runge-Kutta 4 integrator analyses the slope four time -

1. Once at the beginning
2. Twice at the middle
3. Once at the end

it gives a better estimate of the true trajectory.

Mathematically RK4 has fourth-order accuracy meaning error behaves like $O(dt^4)$.

Since the lorenz system is a dissipative system and the energy in it is not conserved, we can disregard the fact that RK4 doesn't preserve the special structure of the system. We would not be able to do the same in a Hamiltonian system like the three-body problem.

The lorenz equations are as follows:

$$dx/dt = \sigma(y-x)$$

$$dy/dt = x(\rho - z) - y$$

$$dz/dt = xy - \beta(z)$$

How the lorenz system looks depends on the choice of the parameters. To get the infamous lorenz attractor (the butterfly shape), we use the following parameters :

$\text{Sigma} = 10$

$\text{Rho} = 28$

$\text{Beta} = 8/3$

The initial state and time step that I have used here is as follows :

$\text{state} = [1, 1, 1]$

$\text{dt} = 0.01\text{s}$

Using MATPLOTLIB, I got the following graph

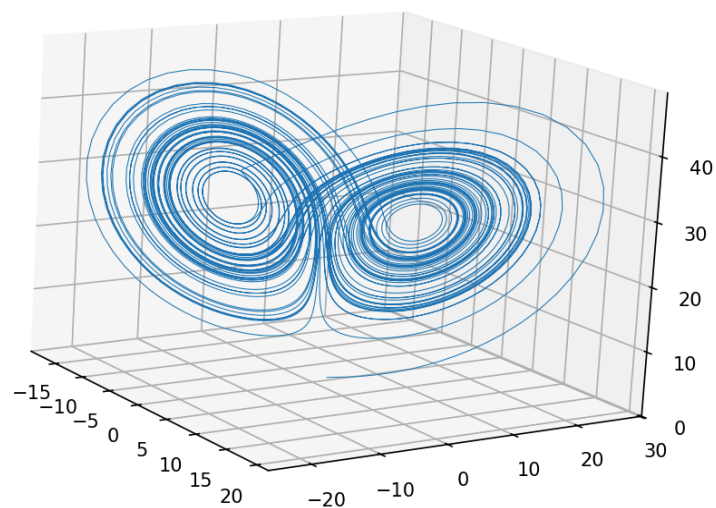


fig 1. Lorenz attractor with $\text{dt} = 0.01\text{s}$

Error - Abnormally large value of distance while calculating the divergence between two lorenz systems

After successfully simulating the lorenz attractor, I then moved on to the second goal which was comparing the deviations that occurred between two lorenz systems with different states, with time. Computing the distance between them at each timestep was what I did next. However, the computed separation came out to be 26 at the start of the simulation.

```
Integrators > lorenz_divergence.csv >
1  t,distance
2  0,26.3325
3  0.01,26.3324
4  0.02,26.3293
5  0.03,26.323
6  0.04,26.3136
```

During the initial development of the Lyapunov exponent calculator, I encountered a significant error where the starting distance between two trajectories was calculated as ~ 26 units, despite an intended initial perturbation of only 10^{-6} .

The Initial Logic (Flawed):

1. Evolve System A for its entire duration ($t=0$ to $t=80$).
2. Start a new loop to evolve System B.
3. Compare the *current* state of System B (t_{current}) with the *final* state of System A (t_{final}).

I realized that for a Lyapunov exponent to be meaningful, the two systems must be compared at the exact same point in time.

In my first attempt, I was inadvertently comparing the starting point of one trajectory with the future endpoint of another. Because the Lorenz attractor is bounded but highly complex, any two points chosen at random (or at vastly different times) will likely be separated by the diameter of the attractor—which in this case was roughly 26 units.

To find λ from my simulation data, I took the natural log of the distance to turn the exponential curve into a straight line:

$$\ln(\Delta Z(t)) = \lambda t + \ln(\Delta Z_0) \quad \text{---(1)}$$

This matches the straight-line equation $y = mx + c$:

- **y-axis:** $\ln(\text{distance})$.
- **x-axis:** time (t).
- **Slope (m):** This is the **Lyapunov exponent (λ)**.

The Estimated Lyapunov exponent came out to be **0.0008416866706356642** in this case which is nearly zero. This implies that the system was behaving non-chaotically during the calculation window. In dynamical systems, a Lyapunov exponent nearly equal to zero implies:

- Unlike a chaotic system where small errors explode quickly, a system with a near-zero exponent remains predictable for a significant amount of time because the "Butterfly Effect" is not yet active.
- In this specific experiment, the near-zero value indicated that the trajectories had already drifted as far apart as the Lorenz attractor allows. Once the distance reaches the attractor's physical bounds, the growth stops, causing the slope and lambda to flatten out.

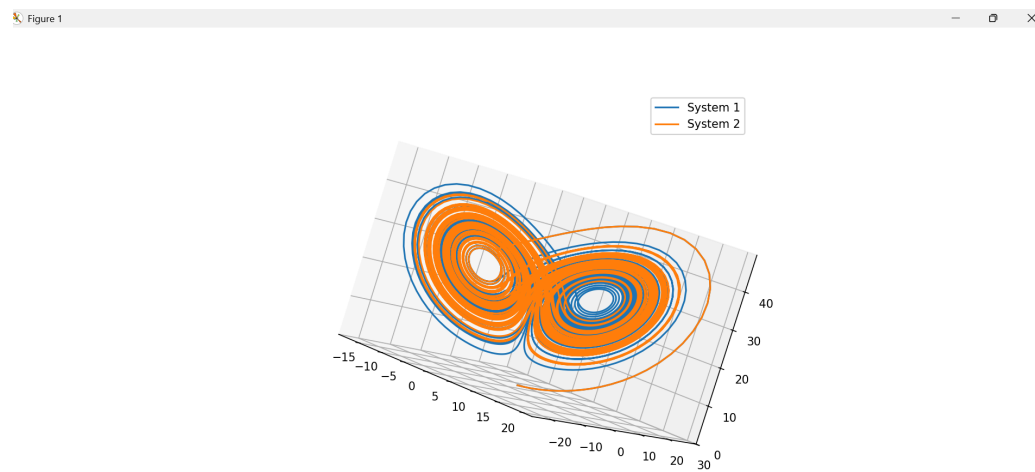


Fig 2.

Comparison between lorenz states with different starting states

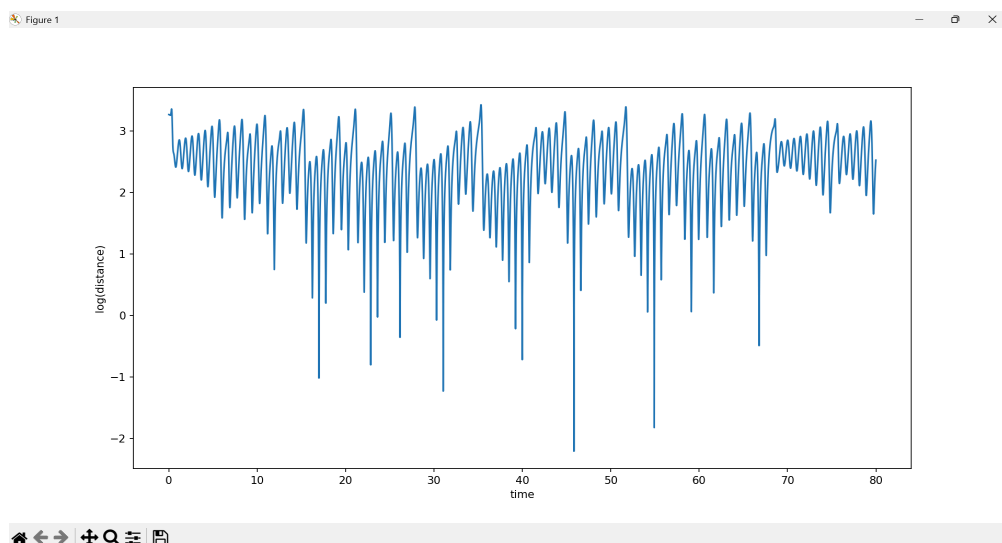


Fig 3. log(distance) vs time

The Fix: Parallel RK4 Integration

To fix the error, I restructured the C++ physics engine to evolve both trajectories simultaneously within a single loop. Originally, the code ran the first system to completion before starting the second, which meant I was comparing a starting point to a future endpoint. By integrating both states at every time step dt , I ensured they were synchronized in time.

This allowed the simulation to correctly capture the initial $10^{(-7)}$ separation and track the true exponential divergence of the system.

In this case the Estimated Lyapunov exponent came out to be **0.18239768943256734**. This value is positive and it signifies that the system is unpredictable over long periods because it has sensitive dependence on initial conditions.

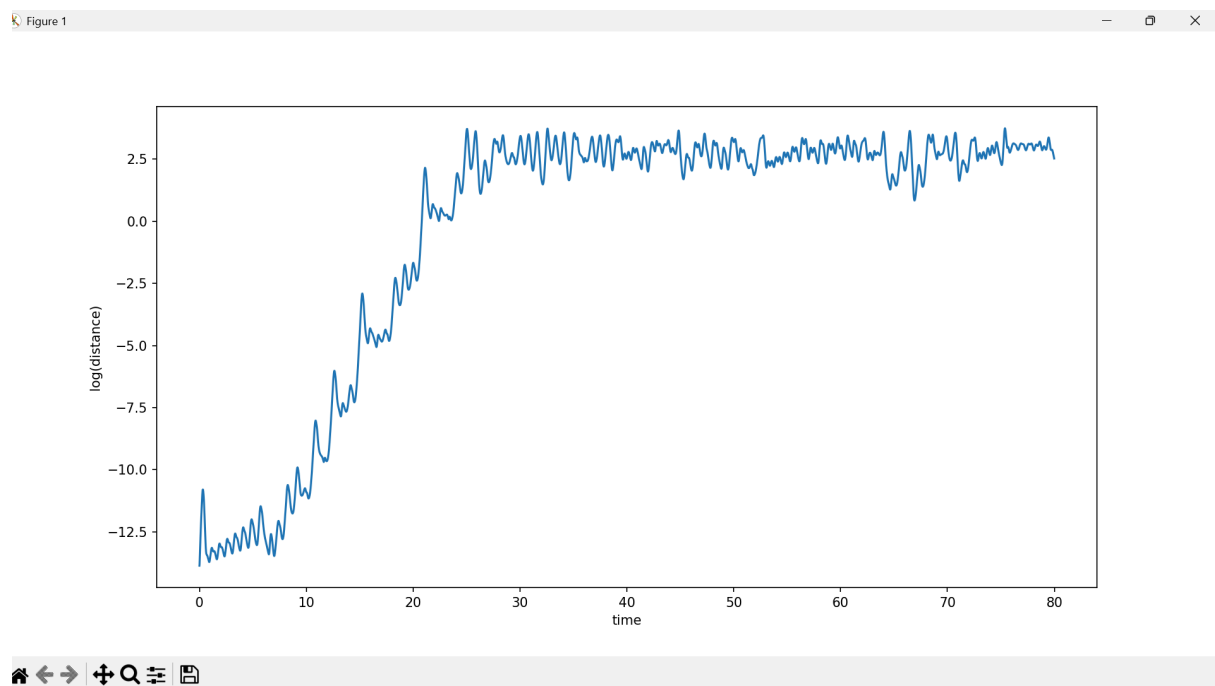


Fig 4. The corrected log(distance) vs time graph

But it is still nowhere close to the actual value of 0.906.

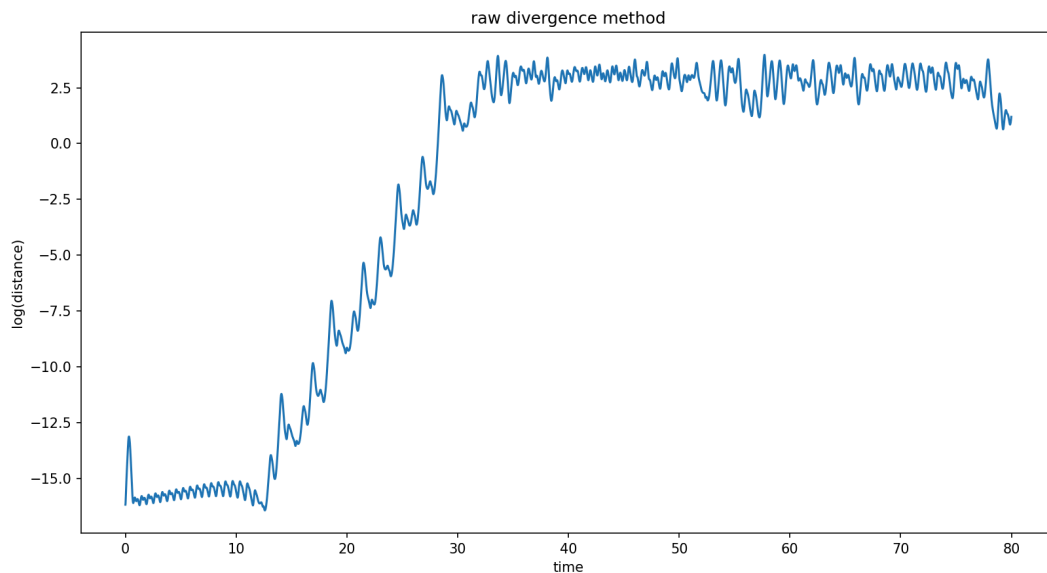
Raw Divergence

Equation 1 does not hold when distance between the two trajectories is small. Here, it becomes very large, leading to the divergence from the standard value.

To rectify this, I implemented the Raw Divergence method again but this time to calculate the lyapunov exponent I used the following formula:

$$\lambda = \lim_{t \rightarrow \infty} \lim_{|\delta_0| \rightarrow 0} \frac{1}{t} \ln \frac{|\delta(t)|}{|\delta_0|} \quad \text{---(2)}$$

Here, I evolved the two trajectories again without any renormalisation.



The log(d) vs time graph remains the same as these values are not changing.

However now, the lyapunov exponent comes out to be the following : 2013.8811

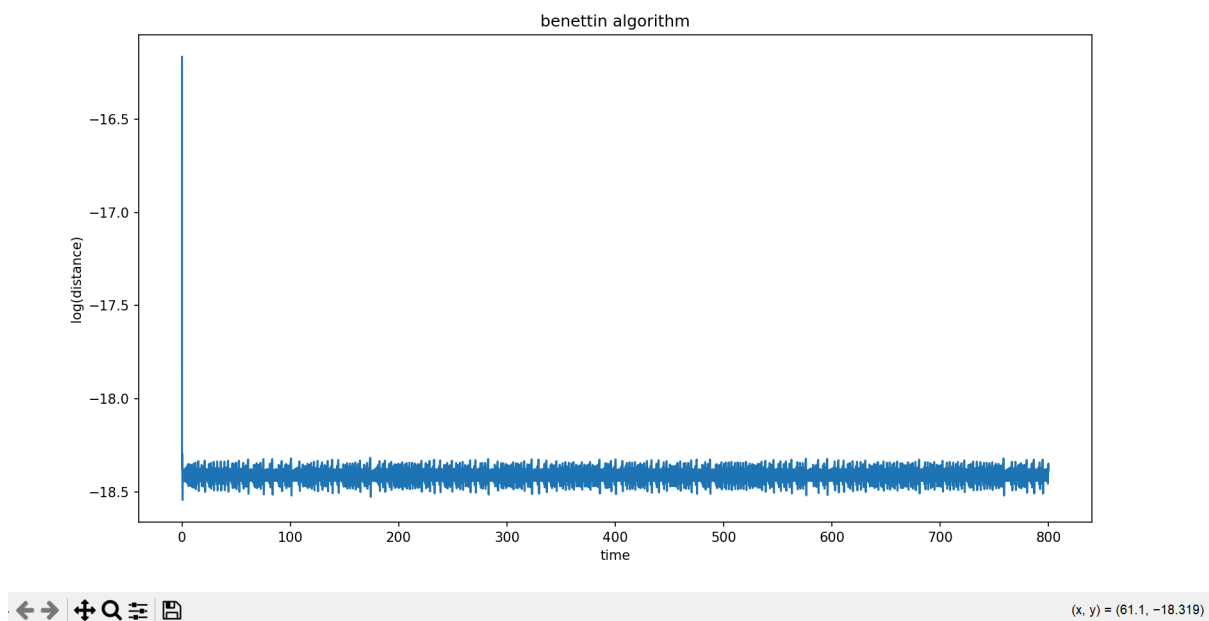
This value is absurd but expected as we didn't renormalise the distance, which causes the difference between the trajectories to become massive as they move to the opposite sides of the attractor.

Benettin with Renormalisation

Now to correct this, I evolved the two trajectories again but this time I used renormalisation of the distance to keep it small enough to keep the system linear. In previous cases, the distance between the two trajectories was increasing such that they were no longer nearby, leading the system to become nonlinear.

I took $d_0 = 10^{-8}$

The $\log(d)$ vs time graph came out to be the following for 800 steps :



I used a python script to find out the lyapunov exponent using formula 2.

The lyapunov exponent is : 0.8224959490590342

This is closer to the standard value of 0.906 but still not quite right. I plan to implement QR decomposition/Jacobian matrix method to further compare the methods for calculation of lyapunov exponent.